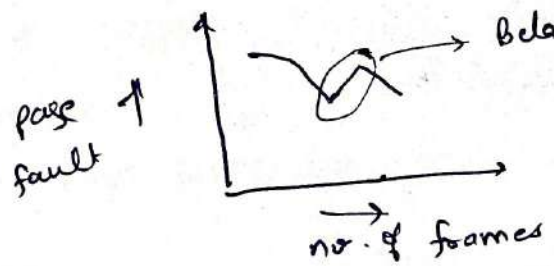
Q1)

	P1	P3	P5	P7	P8
23	Running	Running	Suspended	Ready	Blocked
37	Exited	Exited	~~Exited~~ ~~Running~~ Blocked	Ready	Running
47	Exited	Exited	Exited	Running	Exited

Q2)

Belady's Anamoly is that with the increase in number of frames, page fault increases instead of decreasing. occurs in FIFO.



a) LRU → Ranking = 4.
Belady's Anamoly X
[Replace the page which is least recently used]

b) FIFO → Ranking = 5
Belady's Anamoly ✓
[Replace the page which came first]

c) Optimal → Ranking = 2
Belady's Anamoly X
[Replace the page which will occur after longest time]

d) Second chance → Ranking = 3
Belady's Anamoly X
[Extension of LRU beside in it we give an extra chance]

Q3)

a) LRU Replacement -

1 frame :-

Page faults = 20

2 frame -

Page faults = 18

3 frame -

Page faults = 15

4 frames -

Page faults = 10

5 frames :-

Page faults = 8

6 frames -

Page faults = 7

7 frames -

Page faults = 7

b) Fifo

1 frame :-

Page fault = 20

2 frames :-

Page faults = 18

3 frames -

Page faults = 16

4 frames -

Page faults = 14

5 frames -

Page faults = 10

6 frames -

Page fault = 10

7 frames -

Page fault = 7

c) optimal → Replace that page which will occur after longest time.

1 frame

Page fault = 20

2 frame

Page faults = 16

3 frame

Page faults = 11

4 frames

Page fault = 8

5 frames

Page fault = 7

for 6 frames, 7 frames also

Page faults = 7

Q4)

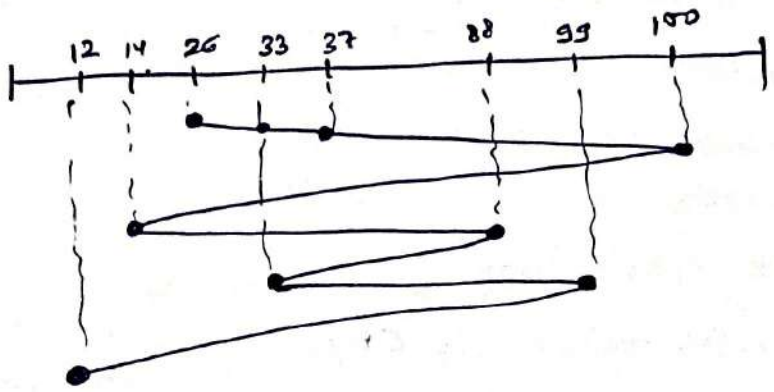
a) yes both the process will be blocked forever if we run them simultaneously.

Consider that both processes start together then foo will execute semWait(S) where it will make S-- i.e S=0. Also at the same time bar will execute semWait(R) where it will make R-- i.e R=0. So when both process reaches to semWait(R) and semWait(S) respectively then both will get struct in infinite loop of "while(R≤0);" and "while(S≤0);" respectively. So they will get blocked forever.

b) No this cannot happen because consider the case when one of the two process goes in indefinite postponement then either S≤0 or R≤0 or both. In that case other will also get struck in semWait(S) or semWait(R). So if one of them is running it implies S=1, R=1 then other will also run. One cannot get indefinitely postponed.

Q5)

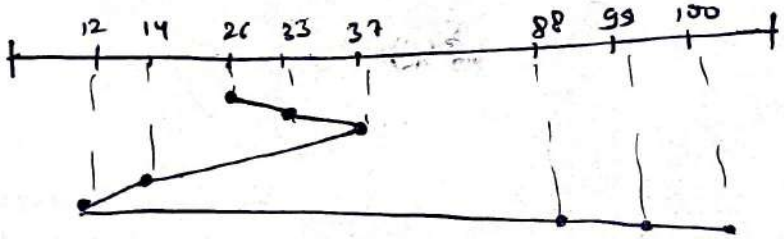
(a.) First come first serve.



Total head movement = 74 + 86 + 74 + 55 + 66 + 87

= 442

(b) SSTF

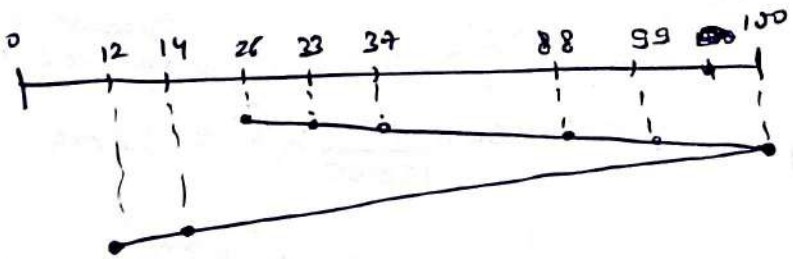


Total head movement = 11 + 25 + 88

= 124

Assumption –

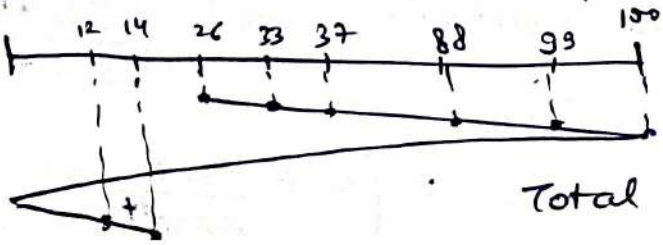
(c) SCAN – Taking maximum to be 100 only and minimum to be 0.



Total head movement = 74 + 88

= 162

d) C-SCAN → same assumption as prev. part.



Total head movement = 74 + 100 + 14
100

Q6)

Rotational speed = 15000 rpm

512 bytes / sector

400 sector / track

Total tracks = 10000

Average seek time = 4ms

Assumption → Transfer rate = x GB/s

(a) Transfer time for 1MB –

$$\Rightarrow \frac{1 \times 10^{10} \times 10^{10}}{x \times 10^{30}}$$

$$\Rightarrow \frac{1}{x \times 10^{10}} \text{ seconds}$$

$$\Rightarrow \frac{1}{x \times 10^{7}} \text{ ms}$$

(b) Average Access time = Average seek time + Rotonal latency (average)

$$= 4\text{ms} + \left(0.5 \times 1000 \times \frac{60}{15000}\right)\text{ms}$$

$$= 4\text{ms} + 2\text{ms}$$

$$= \underline{6\text{ms}}$$

(c) Rotational delay: $0.5 \times 1000 \times \frac{60}{15000} \times 6 = 12\text{ms}$

(Because to read 1 MB we need to read from 6 tracks)

d) Total time to read 1 sector = (Average Access time) $6\text{ms} +$ (transfer time) $\left(\frac{512}{x \times 10^{30}}\right)$ sec

$$= \left(6 + \frac{512}{x \times 10^{37}}\right)\text{ms}$$

e) Total time to read 1 track $= 6 + \frac{512 \times 400}{x \times 10^{30}} \times 1000$

$= \left(6 + \frac{2048}{x \times 10^{25}}\right)$ ms

Q7)

Each block pointer identifies one block of 8KB.

File size $\Rightarrow \left[(13 \times 8) + (1 \times 8) + \frac{(2^{16} \times 8)}{32} + \left(\frac{2^{16}}{32} \times \frac{2^{16}}{32} \times 8\right)\right]$ KB

(13 × 8): 13 direct pointer; (1 × 8): 1 indirect pointer; $\frac{(2^{16} \times 8)}{32}$: 1 double indirect pointer; $\left(\frac{2^{16}}{32} \times \frac{2^{16}}{32} \times 8\right)$: 1 triple indirect pointer

$= \left\{104 + 8 + \frac{2^{16}}{4} + \frac{2^{32}}{32 \times 4}\right\}$ KB

$= \left(112 + \frac{2^{16}}{4} + \frac{2^{30}}{32}\right)$ KB

$= 112 + \frac{2^{16}}{4} + 2^{25}$

$= 112 + \frac{2^{16}}{4} + 2^{20} \times 32$

$2^{20} \times 32$: 32 MB

$= 112\text{KB} + \frac{1}{64}\text{MB}$

$\simeq 32$ MB

Q8)

A lock system can be used where one side cannot create two connection. A variable can be maintained if there is connection from attacker to target then that variable bit is set to 1 and if variable bit is 1 then a new connection cannot be started. But in this case we need to assure one thing that the challenge send by target is one-time challenge and it expires as the connection is lost. If the target sends same challenge again and again then the attacker will initiate a connection and after getting challenge it will disconnect and send the challenge to target as its own and obtain sol^n and again initiate connection where target sends the challenge and attacker gives solution and game over. So we need two things that there is one to one connection only i.e. there can be only one connection between two parties and another thing is that challenge is one time it gets over as the connection get lost. So in this way target can be guarded from such an attack.